

PYTHAI / DELTAVERSE / BANKON — Cross-Chain Wrapped Asset Architecture Research

Apache 2.0 — (c) 2026 BANKON — all rights reserved.

TL;DR

- Kraken deprecated its LayerZero OFT integration for kBTC on May 14, 2026 in favor of Chainlink CCIP and the Cross-Chain Token (CCT) standard, directly motivated by the April 18, 2026 KelpDAO ~\$292M LayerZero-DVN single-verifier exploit; the architectural lesson is that any wrapped-asset platform must structurally eliminate app-configurable single-verifier paths.
- Every multi-hundred-million bridge exploit since 2021 reduces to one of five attack classes — validator/MPC key compromise, signature/proof verification bypass, faulty initialization or replay, admin/keeper substitution, and ULNv2 misconfiguration — and a defense-in-depth design that requires *all* of Wormhole NTT's 13-of-19 Guardian quorum, Pyth pull-oracle price attestation, BONAFIDE-Senatus 5-of-7 sovereign veto and per-asset rate limiters dominates both LayerZero and CCIP on attacker work-factor.
- The recommended Pyth Wrapped Asset Factory (PWAF) deploys an Algorand sovereign layer first, an EVM economic layer second, openBDK as the BFT bridge, x402-avm for per-call metering (USDC ASA 31566704), and CAIP-2 routing (Algorand mainnet `wGHE2Pwvdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkIt8=`); the attacker work-factor is the *product* of four independent compromise events versus a single DVN compromise sufficient under LayerZero or DON+RMN under CCIP. [GitHub](#)

Key Findings

The Kraken-LayerZero deprecation announcement (May 14, 2026) is the cleanest market signal yet that app-configurable verifier sets are commercially unacceptable for institutional wrapped assets. Kraken explicitly stated it was "deprecating its existing cross-chain provider and migrating to Chainlink CCIP as its exclusive cross-chain infra to secure Kraken Wrapped Bitcoin (kBTC) and all future Kraken Wrapped Assets" (Kraken via X, May 14 2026; reported by Chainlink Today, CoinDesk, The Block, Crypto Briefing, Blockonomi). The cited technical rationale was ISO 27001 and SOC 2 Type 2 attestations, the independent Risk Management Network, at least 16 CCIP node operators per lane, native rate limiters, and the audited Cross-Chain Token (CCT) standard. A Chainlink spokesperson quoted in The Block (May 14, 2026) stated that over \$3 billion in TVL has flowed to Chainlink in recent weeks, with Kelp DAO, Solv Protocol and Re alone representing a combined \$2.57 billion of that figure per The Block's own tally. [Crypto Times + 7](#)

The technical comparison shows three distinct trust regimes: LayerZero is permissionless and sovereignty-friendly but *security-weak by default* (the app picks the DVN set, KelpDAO picked 1-of-1, and Lazarus drained 116,500 rsETH on April 18 2026); CCIP is *security-strong but sovereignty-weak* (Chainlink Labs operates the TokenAdminRegistry root and the DON is reputationally opaque); the PWAF design proposed here is both — it inherits Wormhole NTT's 13-of-19 reputation Guardian quorum, layers Pyth pull-oracle deviation envelopes on top, and gates every wrap/unwrap with a BONAFIDE-Senatus sovereign veto rooted in the user's own DAO. No bridge exploit in the 2021–2026 catalog (Ronin, Wormhole, Poly, Nomad, BNB Token Hub, Harmony, Multichain, Mixin, Orbit, HTX/Heco, KelpDAO) would have succeeded against the PWAF gate stack because each was defeated by exactly one mode of attack, while PWAF requires the *intersection* of four independent compromises. (Blockonomi)

The condensed exploit class catalogue is: (1) validator or MPC key compromise — Ronin (\$625M, 5-of-9 PoA, Lazarus spear-phish), Harmony Horizon (\$100M, 2-of-5 plaintext keys), Multichain (\$126M, MPC threshold compromise), Orbit (\$81.5M, 7-of-10 multisig compromise), HTX/Heco (\$86.6M, operator key); (2) signature verification bypass — Wormhole (\$325M, Solana `verify_signatures` accepted a forged sysvar account because the deprecated `load_instruction_at` did not check the address); (3) proof-library bug — BNB Chain Token Hub (~\$570M, IAVL right-child not checked, forged Merkle leaf from a two-year-old proof), Nomad (\$190M, `committedRoot = 0x00` + `confirmAt[0x00] = 1` auto-proved every message); (4) admin/keeper substitution — Poly Network (\$611M, sighash collision `f1121318093` ↔ `putCurEpochConPubKeyBytes` both hashing to 0x41973cd9 letting EthCrossChainManager rewrite the EthCrossChainData Keeper); (5) app-configurable verifier — KelpDAO via LayerZero (\$292M, 1-of-1 DVN). Class (6), off-chain infra compromise, captures Mixin Network (~\$200M, cloud DB compromise). (GitHub + 14)

Details

Part 1 — Kraken kBTC

kBTC is a 1:1 BTC-backed ERC-20 issued by Kraken Financial, the Wyoming-chartered Special Purpose Depository Institution that holds the BTC backing. The Ethereum mainnet token is a transparent upgradeable proxy at `0x73E0C0d45E048D25Fc26Fa3159b0aA04BfA4Db98` (`KBTCProxy`), Solidity 0.8.24, OpenZeppelin v5 `TransparentUpgradeableProxy`, pointing at the implementation `KBTCV2` at `0x16ebc7f9db34f049fe6c6d10250f7d597eca96f1` (previously at `0x4bb46fdb7f47524a9441e6c835a032be257f49e1`). Etherscan's overview text confirms "kBTC is a wrapped Bitcoin token issued by Kraken. It's fully backed by Bitcoin held in Kraken's custody and designed for use on Ethereum and OP Mainnet (and more to come)." kBTC was launched in October 2024 on Ethereum and OP Mainnet, then extended to Ink, Unichain, and Optimism. Its market capitalization as of mid-May 2026 was \$260 million, per CoinGecko data cited by CoinDesk: "Kraken introduced kBTC in 2024 as a 1:1 bitcoin-backed token available first on Ethereum and OP Mainnet. The token now has a \$260 million market capitalization, CoinGecko

data shows." DefiLlama TVL for kBTC at the time was approximately \$333M according to The Block, reflecting deposit balances rather than circulating-supply-times-spot. Kraken's whitepaper explicitly acknowledges that "Kraken effectively controls token management functions through a Kraken-controlled wallet." (Crypto Briefing + 7)

The original LayerZero OFT integration (announced in 2025 by LayerZero and Kraken) standardized kBTC on the Omnichain Fungible Token standard so the same canonical kBTC could traverse the 150+ chains LayerZero supports without separate per-pair bridges. The OFT model burns kBTC on the source chain and mints on the destination, with the proof of state carried by a LayerZero packet whose verification is delegated to an OApp-configured Security Stack: required DVNs, optional DVNs, and an optional threshold. The LayerZero v2 Ultra Light Node (ULN) message library supports up to 254 DVNs. The Endpoint marks the ((nonce, payloadHash)) pair as Verified once required + threshold optional DVNs sign via secp256k1 multisig, and the Executor then submits the packet to the destination. The KelpDAO breach exposed the fundamental flaw: because the OApp picks the DVN set, an OApp can pick a 1-of-1 DVN — which is exactly what KelpDAO did, and which Lazarus exploited on April 18, 2026 to forge a Unichain-origin packet that the single DVN signed, after which the Ethereum OFTAdapter released 116,500 rsETH (~\$292M). LayerZero stated post-hoc that allowing its own verifier network to secure such high-value assets in that configuration was "a mistake."

(LayerZero + 5)

On May 14, 2026 Kraken announced it would deprecate LayerZero and migrate kBTC and all future Kraken Wrapped Assets to Chainlink's Cross-Chain Interoperability Protocol (CCIP) as the *exclusive* cross-chain infrastructure. The migration is intended to cover Ethereum, Optimism, Ink and Unichain initially, with further chains to follow. The CCT installation procedure for kBTC is the same as for any CCIP-enabled ERC-20: the token must expose (getCCIPAdmin() returns (address)); a token pool (typically (BurnMintTokenPool) v1.5.1 for native multi-chain assets) is deployed per chain with constructor ((IBurnMintERC20 token, uint8 localTokenDecimals, address[] memory allowlist, address rmnProxy, address router)); administrators register the token/pool pair in (TokenAdminRegistry) through (RegistryModuleOwnerCustom.registerAdminViaGetCCIPAdmin(token)) followed by a two-step (proposeAdministrator) / (acceptAdminRole) / (setPool) flow; cross-chain transfers thereafter use (IRouterClient(router).ccipSend(uint64 destinationChainSelector, Client.EVM2AnyMessage memory)) where (EVM2AnyMessage) carries ({bytes receiver; bytes data; Client.EVMTokenAmount[] tokenAmounts; address feeToken; bytes extraArgs}) and (extraArgs) is (Client._argsToBytes(Client.EVMExtraArgsV1({gasLimit: ...}))). On the destination, the CCIP (OffRamp) calls (balanceOf) → (releaseOrMint(Pool.ReleaseOrMintInV1)) → (balanceOf) against the local pool, with a default 90,000-gas budget across those three calls. The Risk Management Network is a separate Decentralized Oracle Network running OCR2 consensus that independently veto-signs every CCIP commit; this two-network split is the structural property Kraken cited. The canonical Foundry pin from the Chainlink docs is (forge

`install smartcontractkit/chainlink-ccip@2114b90f39c82c052e05af7c33d42c1ae98f4180`].

(MEXC + 6)

For future Kraken wrapped assets, Kraken stated the CCT/CCIP migration "covers kBTC and all future Kraken Wrapped Assets." No specific kETH, kSOL, kUSD, or RWA wrappers have been publicly announced as of May 2026; the parent company Payward applied this month for a federal trust charter, signalling RWA pipelining. The CCT pattern generalizes trivially to any ERC-20 with `getCCIPAdmin()` plus mint/burn or lock/release semantics. (The Block) (CoinDesk)

Part 2 — Cross-Chain Bridge Exploit Catalog

Poly Network — August 10, 2021 — ~\$611M — admin / keeper substitution. Poly's cross-chain `EthCrossChainManager` owned `EthCrossChainData`. Manager's `verifyHeaderAndExecuteTx → _executeCrossChainTx` could call any target; the attacker brute-forced the function name `f1121318093(bytes,bytes,uint64)` whose first-four-byte selector is `0x41973cd9`, identical to that of `putCurEpochConPubKeyBytes(bytes)`. Manager invoked Data's privileged setter and the attacker became sole Keeper across Ethereum, BSC and Polygon. Postmortems: Kraken Security Labs ("Abusing Smart Contracts to Steal \$600 million"), SlowMist, BlockSec, Kudelski Security, rekt.news. The funds were largely returned by the attacker. (Kudelski Security Research) (Kudelski Security Research)

Wormhole — February 2, 2022 — ~\$325M (120,000 wETH) — signature verification bypass. Solana's `verify_signatures` used the deprecated `solana_program::sysvar::instructions::load_instruction_at` without validating that the passed sysvar account was the canonical `SysvarInstructions11111111111111111111111111111111`. The attacker supplied a controlled account `2tHS1cXX2h1KBEaadprqELJ6sV9wLoaSdX68FqsrrZRd` preloaded with a forged Secp256k1 instruction; the resulting `SignatureSet` satisfied `post_vaa`; `complete_wrapped` minted 120,000 wETH on Solana with no ETH locked. Jump Trading replenished the bridge from its own balance sheet. Postmortems: Halborn ("Explained: The Wormhole Hack"), CertiK, BlockSec, Kudelski Security, TRM Labs, Merkle Science. (LinkedIn + 3)

Ronin Bridge — March 23, 2022 — \$625M (173,600 ETH + 25.5M USDC) — validator key compromise. Sky Mavis's Ronin used a 5-of-9 Proof-of-Authority validator set. Lazarus spear-phished a Sky Mavis engineer through a fake LinkedIn job offer culminating in a malware-laden PDF; the attacker compromised the four Sky Mavis validator keys and obtained the fifth (Axie DAO) signature via a still-allowlisted gas-free RPC node from a November 2021 event. The exploit was only noticed six days later. Postmortems: Ronin chain blog ("Back to Building"), Elliptic, tayvano/lazarus-bluenoroff-research, Bankless ("Analyzing the Ronin bridge hack"). Sky Mavis has since increased the validator set to 22 nodes. (Grvt + 4)

Harmony Horizon Bridge — June 23, 2022 — \$100M — 2-of-5 multisig compromise. The bridge was secured by a 2-of-5 hot-wallet multisig. Two signer keys were compromised — likely stored in plaintext on infected servers for processing legitimate bridging transactions. The

attacker drained ETH, WETH, WBTC, USDT, USDC, BUSD and DAI across 14 transactions on Ethereum and BSC, then laundered through Tornado Cash and Railgun. Postmortems: Mudit Gupta (Polygon CSO) Twitter thread, Elliptic, Merkle Science, harmony.one incident report.

(LimeChain + 4)

Nomad Bridge — August 1, 2022 — \$190M — zero-root initialization / Merkle bypass. A routine upgrade of the Replica contract set `committedRoot = 0x00` and `confirmAt[0x00] = 1`. Because newly proven messages stored their root in a mapping whose default value is `0x00`, any unprocessed message was automatically deemed proven by the `process()` function. samczsun's tl;dr: "a routine upgrade marked the zero hash as a valid root, which had the effect of allowing messages to be spoofed on Nomad." The first attacker drained 100 WBTC; within hours, 300+ "crowd-looters" copy-pasted the calldata with their own recipient and emptied the bridge. Postmortems: Immunefi ("Hack Analysis: Nomad Bridge"), CertiK, Mandiant ("Decentralized Robbery"), Solidgroup. Approximately \$37M was returned by white hats. (BankInfoSecurity + 2)

BNB Chain BSC Token Hub — October 6, 2022 — ~\$570M (2M BNB) — IAVL Merkle proof forgery. The Cosmos `iavl` library's range-proof validator did not reject internal nodes that had a `Right` child whose hash matched a leaf's hash; this meant an injected right leaf containing the SHA-256 of an attacker payload did not affect the computed root. The attacker deposited 100 BNB to become a BSC bridge relayer, replayed a two-year-old legitimate proof from block 110217401, injected a forged leaf into the right field with the hash of a payload minting 1M BNB to themselves, and called `handlePackage` twice. BNB Chain validators halted the chain roughly 90 minutes after, freezing about \$400M; about \$137M escaped via Stargate/Anyswap to Ethereum, Avalanche, Polygon, Fantom and Optimism. The Moran hardfork at block 22,107,423 patched the IAVL hash check, added sequential block-header checks, and whitelisted relayers. Postmortems: Halborn, Immunefi ("Hack Analysis: Binance Bridge"), Coinbase Threat Intel ("BSC Token Hub Compromise Investigation"), QuillAudits, Nansen, PT Swarm. (Halborn + 6)

Multichain — July 6-7, 2023 — \$126M — MPC threshold compromise. Multichain (née AnySwap) used an MPC scheme. Assets worth roughly \$118M were drained from the Fantom bridge contract on Ethereum (\$58M USDC + \$13.6M WETH + \$30.9M WBTC + ~\$16M in DAI/LINK/USDT), plus \$6.8M from the Moonriver bridge and \$0.6M from Dogechain. CEO Zhao Jun had been arrested in China; the team lost contact and access to MPC key shares; CertiK confirmed private-key compromise outside its audit scope. The Singapore court appointed a liquidator. Postmortems: Chainalysis ("Multichain Exploit"), CertiK, Secure3, Neptune Mutual.

(Substack + 7)

Mixin Network — September 23, 2023 — ~\$200M — cloud database compromise of Mixin's cloud service provider. (Substack)

HTX/Heco Bridge — November 22, 2023 — \$86.6M — operator key compromise (plus \$12M HTX hot wallet drain on the same day). (Hacken)

Orbit Chain — December 31, 2023 — \$81.5M — multisig compromise (reportedly 7 of 10 multisig signatories compromised). [COIN360](#) [CoinPedia](#)

KelpDAO via LayerZero — April 18, 2026 — \$292M (116,500 rsETH) — single-DVN compromise. KelpDAO's OFTAdapter on Ethereum was configured with a single mandatory DVN (1-of-1). Lazarus compromised that DVN's signer set, forged a Unichain-origin packet, the DVN signed, OFTAdapter released. A second packet for 40,000 rsETH was authenticated but blocked by KelpDAO's emergency multisig. Postmortem: blockaid.io ("How a Single LayerZero DVN Compromise Drained \$292M from KelpDAO"). LayerZero acknowledged the configuration error. [Blog + 2](#)

Other notable incidents catalogued by class but with smaller losses: Chainswap (July 2021, \$4.4M, unprotected mint), AnySwap (July 2021, \$7.9M, ECDSA r-nonce reuse permitting private-key recovery), pNetwork (September 2021, \$12M, malicious pegIn parsing), Qubit Finance (January 2022, \$80M, fake [WithdrawalRequested](#) via [safeTransferFrom](#) on the zero address), Meter.io (February 2022, \$4.4M, missing native value check), EvoDeFi (2022, depeg, no PoR), Allbridge (April 2023, \$573K, oracle manipulation via swap), Thorchain (2021 incidents totaling ~\$13M, refund logic + ETH router bugs).

The attack-class taxonomy for systematic defense is: (1) validator/MPC key compromise (Ronin, Harmony, Multichain, Orbit, HTX, Mixin, KelpDAO); (2) signature verification bypass via unchecked input (Wormhole, Qubit); (3) proof library / Merkle bug (BNB IAVL, Nomad zero-root); (4) privileged target via cross-chain relay / sighash collision (Poly Network); (5) app-configurable verifier with insufficient quorum (KelpDAO); (6) operational / cloud / supply chain (Mixin DB, AnySwap r-nonce); (7) front-end / approval phishing (Squid Router, Atomic Wallet — out of bridge scope).

Part 3 — Cross-Chain Messaging Layer Comparison

LayerZero v2 / ULNv2 / DVN / OFT. Trust model is 1-of-N where N is application-chosen. Each OApp configures a Security Stack via [EndpointV2.setConfig](#) containing required DVNs, optional DVNs and a confirmation threshold per pathway. The ULN supports up to 254 DVNs and each implements [ILayerZeroDVN.assignJob\(...\)](#) / [getFee\(...\)](#), signing the source-chain [payloadHash](#) via secp256k1 multisig. The failure mode is structural — app developers can pick weak or single DVNs — and this is exactly what produced the KelpDAO loss. LayerZero is permissionless and sovereignty-friendly, but is *security-weak by default*. [Blockaid + 2](#)

Chainlink CCIP / DON / RMN / OCR2. Trust model is t-of-n honest in the committing DON (≥ 16 nodes per lane) plus an independent Risk Management Network running OCR2 that veto-signs every commit. Per-token, per-direction rate limiters bound damage. To mint counterfeit tokens an attacker must compromise $\geq t$ of the committing DON *and* defeat the RMN veto *and* exceed the rate limiter inside its refill window. CCIP is sovereignty-weak (Chainlink Labs operates the TokenAdminRegistry root) but security-strong. [Blockonomi](#)

Wormhole Guardian Network / NTT. Trust model is 13-of-19 honest Guardians selected by reputation (Jump, Certus One, Chorus One, Everstake, Figment, P2P, Staked, others). VAAs are secp256k1 multisignatures over a `(keccak256)` hash of the message body `{version, guardian_set_index, timestamp, nonce, emitter_chain, emitter_address, sequence, consistency_level, payload}`. Replay protection is per-chain `consumedVaa[hash] = true`. NTT introduces `NttManager` (one per token, owning lock/burn logic, rate limits and message attestation) and one or more `Transceiver`s implementing `ITransceiver` (`sendMessage`, `quoteDeliveryPrice`); the default is `WormholeTransceiver` calling Wormhole Core, but custom transceivers can be aggregated for M-of-N attestation (Pi Squared's VSL is a deployed example of a custom zk-attestation route). NTT supports *locking* mode (one canonical chain holds supply) and *burning* mode (distributed natively multichain). Wormhole Queries provide synchronous cross-chain reads via 13-of-19 signed RPC results. Delegated Guardian Sets allow per-chain observation by a subset while the final VAA remains a 13-of-19 multisig. (Marco Hextor + 5)

Axelar. Trust model is BFT PoS on a governance-capped active set of exactly 75 validators, per Messari's *Understanding Axelar*: "Only the top 75 validators are in the active set, a parameter that can be adjusted through onchain governance." As of April 2026 approximately 70 were active per the Mintsan Axelar validator dashboard. Cross-chain calls use General Message Passing through `gateway.callContract` / `executeWithToken`. The failure mode is validator collusion above the 2/3 threshold or governance capture. (Messari) (Eco)

Hyperlane. Trust model is sovereign — each application installs its own Interchain Security Module (ISM); the default `MultisigISM` is t-of-n, and `AggregationISM` / `RoutingISM` can compose multiple ISMs. Hyperlane's permissionless deploy story is the strongest, but the same caveat as LayerZero applies — apps can pick weak ISMs.

Pyth Pull Oracle. Documented under Part 4.

Pyth Entropy. Two-party commit-reveal randomness. The contract on Optimism is

`0xdF21D137Aadc95588205586636710ca2890538d5`, default provider

`0x52DeaA1c84233F7bb8C8A45baeDE41091c616506`. Independent of Wormhole. (Pyth Network)

Pyth Lazer. Ultra-low-latency oracle with 1ms / 50ms / 200ms cadences carrying bid/ask/market depth, deployed on EVM + SVM. Reported latency vs. centralized exchange spot prices is competitive at sub-100ms — PolynomialFi (via blockchain.news) stated that "the integration of Pyth Network Lazer has reduced their AMM latency from 2 seconds to a few hundred milliseconds," and Pyth's own publication of Lazer benchmarks indicates end-to-end latency vs. Binance spot BTC of approximately 70ms at p99. (Crypto News Australia + 2)

IBC (Cosmos). Light-client based per-chain pairing; trust is as strong as the counterparty's consensus. Failure modes: client-freeze attacks, IAVL library bugs (the same library implicated in BSC Token Hub), governance freezes.

zkBridges (Polyhedra zkBridge, Succinct, =nil; Foundation, Herodotus). Trust model is cryptographic — a zk-SNARK/STARK proves the source chain state transition or block header validity. Theoretically trustless (1-of-n liveness only). The practical exploit surface is the admin/upgrade path that controls verifier-key replacement, not the cryptography.

Part 4 — Pyth Native Capabilities

Pyth's pull oracle aggregates first-party publisher prices on Pythnet, a Solana Permissioned Environment (SPE). Pyth has 138 first-party publishers, per the Pyth Network's own KPIs page, which also notes that "Over 120 financial institutions—including some of the world's biggest exchanges, market makers, and trading firms—publish their data directly to the network."

Pythnet's Oracle Program aggregates publisher prices every 400ms into a Merkle root.

Wormhole Guardians observe Pythnet and sign the root as a VAA. Hermes

(<https://hermes.pyth.network>) is the open-source service that listens for these VAAs and exposes REST ([/v2/updates/price/latest?ids\[\]={...}](/v2/updates/price/latest?ids[]={...})) and WSS endpoints. On-chain consumers fetch updates from Hermes and call `IPyth.updatePriceFeeds{value: fee}(bytes[] updateData)` (the fee being `IPyth.getUpdateFee(updateData)`); reads use `getPriceUnsafe(bytes32 id)` or `getPriceNoOlderThan(bytes32 id, uint age)` (which reverts with `StalePrice` selector `0x19abf40e` if older than the requested age). The verbatim Solidity signatures from `pyth-network/pyth-sdk-solidity/IPyth.sol` (Apache-2.0, `pragma solidity ^0.8.0`): `Pyth Network + 8`

solidity

```
function updatePriceFeeds(bytes[] calldata updateData) external payable;
function getPriceUnsafe(bytes32 id) external view returns (PythStructs.Price memory)
function getPriceNoOlderThan(bytes32 id, uint age) external view returns (PythStruct
function getUpdateFee(bytes[] calldata updateData) external view returns (uint feeAm
function parsePriceFeedUpdates(
    bytes[] calldata updateData, bytes32[] calldata priceIds,
    uint64 minPublishTime, uint64 maxPublishTime
) external payable returns (PythStructs.PriceFeed[] memory priceFeeds);
```

`PythStructs.Price { int64 price; uint64 conf; int32 expo; uint publishTime; }` —
real value = `price * 10^expo`. [GitHub](#) [GitHub](#)

Pyth's Wormhole-based message verification on EVM works by Pyth's deployed `PythUpgradable` contract holding the canonical Wormhole Core address, the trusted Pyth emitter chain ID 26 and emitter address, and the Guardian set index; `updatePriceFeeds` parses each VAA via `wormhole.parseAndVerifyVM`, checks `vm.emitterChainId == 26` and the emitter matches, extracts the Merkle root from the VAA payload, and verifies the inclusion proof for each

requested feed against that root. Replay protection comes from Wormhole's consumed-VAA mapping plus a monotonic `publishTime` per feed.

Proof-of-reserves with Pyth: Pyth has explicitly been integrated for BTCfi PoR alongside Chainlink PoR feeds. The pattern is — an off-chain attestor publishes a JSON manifest `{root, totalReserves, height}`; a Pyth publisher posts the manifest as a structured feed; consumers verify with `getPriceNoOlderThan` plus a domain-specific Merkle inclusion. This same primitive backs PWAF's `ProofOfReserves` contract.

Pyth on Algorand: there is no canonical Pyth Algorand mainnet deployment. The Pyth contract-addresses page (`docs.pyth.network/price-feeds/core/contract-addresses`) enumerates EVM, Solana/SVM, Aptos, Sui, IOTA, Movement, TON, Fuel, CosmWasm, NEAR, Starknet, Pythnet — but not Algorand. The `pyth-network/pyth-crosschain` monorepo has no `target_chains/algorand` directory. A 2022 Pyth blog post listed Algorand among "up next" chains but that integration has not landed as of May 2026. The Wormhole Algorand core (`wormhole-foundation/wormhole/tree/main/algorand`) does exist and implements `wormhole_core.py`, `token_bridge.py`, `vaa_verify.py` and a `TmplSig.py` stateless logic signature for VAA-sequence dedup via a 2K-bit bitfield. PWAF therefore builds its Pyth Algorand client from first principles, layering Pyth payload parsing on top of the existing Wormhole AVM core. [Pyth Network](#)

The Wormhole Guardian set comprises 19 Guardians at threshold 13. Per `wormhole.com/docs/protocol/infrastructure/guardians/`: "A quorum of 13 signatures is required to produce a valid VAA. ... 19 Guardians form the canonical set." Recent governance introduced Delegated Guardian Sets where a per-chain subset performs direct on-chain observation but the final VAA still requires 13 signatures. The `WormholeDelegatedGuardians` governance contract manages thresholds. The Global Accountant and Governor mechanisms rate-limit suspicious flows. [Wormhole + 2](#)

Wormhole NTT architecture (deeply): `NttManager` is one-per-token, exposing `transfer(uint256 amount, uint16 recipientChain, bytes32 recipient, ...)` and `redeem(EncodedNttManagerMessage)`. `Transceiver` extends `ITransceiver` with `sendMessage(uint16 recipientChain, ...)` and `quoteDeliveryPrice(uint16)`. Rate limiters live on both source (outbound) and destination (inbound). The reference `WormholeTransceiver` holds an immutable Wormhole Core reference, a `WORMHOLE_CONSUMED_VAAS_SLOT` mapping for replay protection, and supports both Wormhole standard relay and special relay paths. Custom transceivers can be combined for M-of-N attestation (e.g. Pi Squared VSL). [GitHub + 2](#)

Part 5 — PYTH-WRAPPED-ASSET-FACTORY (PWAF) Architecture

5.1 Architectural overview

PWAF is a sovereign, defense-in-depth wrapped-asset factory. An underlying — bitcoin, stETH, USDC, gold, real estate, RWA fund share — is locked or attested on its native venue (an L1 escrow

`AssetVault.sol` for EVM-native ERC-20s; a stateless logsig + ASA escrow on Algorand; an off-chain Merkle attestor with Pyth-signed updates for non-EVM and real-world assets). A `WrappedAsset.sol` ERC-20 (optionally ERC-7857 INFT-compatible for identity-bound positions) is deployed per asset per destination by `WrappedAssetFactory.sol`. Mint and burn on any chain are gated by four orthogonal attestations: a Wormhole NTT VAA proving the lock/burn occurred on source; a Pyth price update bounding the implied asset value within a deviation envelope (the "value sanity check"); a BONAFIDE Senatus governance check confirming neither emergency veto nor identity revocation; per-asset, per-direction rate-limiter consumption plus a circuit-breaker check. All four must pass. Unwrap additionally requires a proof-of-reserves Merkle inclusion against the live attested root.

The trust path defeats each Part 2 class: (1) validator/key compromise requires *all four* layers simultaneously; (2) signature-verification bypass cannot mint because Pyth deviation + Senatus still block; (3) Merkle/proof-library bugs are mitigated because PWAF uses only OZ `MerkleProof.verifyCalldata` with explicit `require(root != bytes32(0))`; (4) privileged-target via cross-chain relay is impossible because `AttestationVerifier` exposes no privileged writer callable from cross-chain; (5) single-verifier configuration is impossible because the Wormhole 13-of-19 quorum is hard-coded; (6) operational compromise is mitigated by x402 metering and BONAFIDE per-call attestation.

5.2 Hybrid trust model — cryptoeconomic argument

An attacker who wants to mint counterfeit PWAF wrapped tokens must accomplish *all* of the following in the same epoch: (i) compromise ≥ 13 of 19 Wormhole Guardian secp256k1 keys, historically never achieved; (ii) defeat Pyth price attestation, either by corrupting the Pyth Wormhole VAA emitter (same 13-of-19 requirement, but with a *distinct* emitter address and the Pythnet OCR2 attestation logic) or by publishing a forged price within `MAX_DEVIATION_BPS` (default 200 bps) of the on-chain moving average; (iii) bypass `SenatusGate.require_approval(bytes32 op)` — a BONAFIDE Senatus 5-of-7 sovereign quorum of Tessera-holders; (iv) not exceed `RateLimiter` capacity or trip `CircuitBreaker` (Pyth deviation $> 5\sigma$, Wormhole emergency shutdown bit, Senatus veto). The work factor is the *product* of each layer's compromise probability, not the union. By contrast LayerZero's KelpDAO required *one* DVN compromise (one layer of defense); CCIP requires DON + RMN compromise (two layers, both controlled by Chainlink Labs). PWAF is structurally more secure than either: hard-coded 13-of-19 quorum eliminates Stargate-style weak-DVN choice, and the BONAFIDE Senatus sovereign veto layer sits *outside* the Wormhole+Pyth substrate so a single-substrate compromise still does not mint.

5.3 Solidity factory contracts

solidity

```

// SPDX-License-Identifier: Apache-2.0
// (c) 2026 BANKON – all rights reserved
pragma solidity 0.8.26;

import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
import {Pausable} from "@openzeppelin/contracts/utils/Pausable.sol";
import {MerkleProof} from "@openzeppelin/contracts/utils/cryptography/MerkleProof.sol";
import {SafeERC20, IERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

import {IPyth} from "@pythnetwork/pyth-sdk-solidity/IPyth.sol";
import {PythStructs} from "@pythnetwork/pyth-sdk-solidity/PythStructs.sol";

interface IWormhole {
    struct VM { uint8 version; uint32 timestamp; uint32 nonce; uint16 emitterChainId; }
    function parseAndVerifyVM(bytes calldata encodedVM) external view returns (VM memory vm);
}

interface ISenatus {
    function isVetoed(bytes32 op) external view returns (bool);
    function quorumApproval(bytes32 op) external view returns (bool);
}

/// @title WrappedAsset
/// @notice ERC-20/ERC-7857-compatible wrapped token, mint/burn gated by the factory
contract WrappedAsset is ERC20, AccessControl {
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
    bytes32 public constant BURNER_ROLE = keccak256("BURNER_ROLE");
    bytes32 public immutable assetId;
    address public immutable factory;
    uint8 private immutable _decimals;

    constructor(string memory n, string memory s, uint8 d, bytes32 id, address f, address ad) {
        _decimals = d; assetId = id; factory = f;
        _grantRole(DEFAULT_ADMIN_ROLE, f);
        _grantRole(MINTER_ROLE, minter);
        _grantRole(BURNER_ROLE, burner);
    }

    function decimals() public view override returns (uint8) { return _decimals; }
    function mint(address to, uint256 amt) external onlyRole(MINTER_ROLE) { _mint(to, amt); }
    function burn(address from, uint256 amt) external onlyRole(BURNER_ROLE) { _burn(from, amt); }
}

```

```

/// @title AttestationVerifier
contract AttestationVerifier {
    IWormhole public immutable wormhole;
    IPyth public immutable pyth;
    uint16 public constant PYTHNET_CHAIN_ID = 26;
    bytes32 public immutable pythEmitter;
    uint64 public maxPriceAgeSec = 60;
    uint64 public maxDeviationBps = 200;
    mapping(bytes32 => bool) public consumedVaa;
    mapping(bytes32 => bytes32) public reserveRoot;

    error InvalidEmitter();
    error ReplayedVaa();
    error PriceDeviationTooHigh();
    error BadMerkleProof();

    constructor(address wh, address py, bytes32 emitter) { wormhole = IWormhole(wh);

    function verifyNttVaa(bytes calldata vaa) external returns (IWormhole.VM memory)
        (IWormhole.VM memory v, bool ok, string memory reason) = wormhole.parseAndVerifyVaa(
            vaa, consumedVaa);
        require(ok, reason);
        if (consumedVaa[v.hash]) revert ReplayedVaa();
        consumedVaa[v.hash] = true;
        return v;
    }

    function verifyPythBound(bytes32 priceId, uint256 referenceX18, bytes[] calldata calldata
        uint256 fee = pyth.getUpdateFee(upd);
        pyth.updatePriceFeeds{value: fee}(upd);
        PythStructs.Price memory p = pyth.getPriceNoOlderThan(priceId, maxPriceAgeSec);
        require(p.price > 0, "neg price");
        uint256 pyX18;
        if (p.expo >= 0) pyX18 = uint256(uint64(p.price)) * (10 ** (18 + uint32(p.expo)));
        else { uint256 d = uint256(uint32(-p.expo)); pyX18 = (uint256(uint64(p.price)) * 10 ** d);
        uint256 diff = pyX18 > referenceX18 ? pyX18 - referenceX18 : referenceX18 - pyX18;
        if (diff * 10_000 / pyX18 > maxDeviationBps) revert PriceDeviationTooHigh();
    }

    function verifyReserve(bytes32 assetId, bytes32 leaf, bytes32[] calldata proof)
        bytes32 root = reserveRoot[assetId];
        require(root != bytes32(0), "no root");
        if (!MerkleProof.verifyCalldata(proof, root, leaf)) revert BadMerkleProof();
    }
}

```

```

}

/// @title RateLimiter (token-bucket with exponential refill)
contract RateLimiter {
    struct Bucket { uint128 capacity; uint128 rate; uint128 tokens; uint64 lastRefill;
    mapping(bytes32 => Bucket) public buckets;
    error RateLimited();

    function configure(bytes32 key, uint128 cap, uint128 ratePerSec) external {
        buckets[key] = Bucket(cap, ratePerSec, cap, uint64(block.timestamp));
    }
    function consume(bytes32 key, uint128 amt) external {
        Bucket storage b = buckets[key];
        uint64 dt = uint64(block.timestamp) - b.lastRefill;
        uint128 refill = uint128(dt) * b.rate;
        uint128 newTok = b.tokens + refill;
        if (newTok > b.capacity) newTok = b.capacity;
        b.lastRefill = uint64(block.timestamp);
        if (newTok < amt) revert RateLimited();
        b.tokens = newTok - amt;
    }
}

/// @title CircuitBreaker
contract CircuitBreaker is Pausable, AccessControl {
    bytes32 public constant BREAKER_ROLE = keccak256("BREAKER_ROLE");
    event Tripped(string reason);
    constructor(address admin) {
        _grantRole(DEFAULT_ADMIN_ROLE, admin);
        _grantRole(BREAKER_ROLE, admin);
    }
    function trip(string calldata r) external onlyRole(BREAKER_ROLE) { _pause(); emit Tripped(r); }
    function reset() external onlyRole(DEFAULT_ADMIN_ROLE) { _unpause(); }
}

/// @title SenatusGate
contract SenatusGate {
    ISenatus public immutable senatus;
    constructor(address s) { senatus = ISenatus(s); }
    function require_approval(bytes32 op) external view {
        require(!senatus.isVetoed(op), "Senatus veto");
        require(senatus.quorumApproval(op), "Senatus quorum");
    }
}

```

```

/// @title ProofOfReserves
contract ProofOfReserves is AccessControl {
    bytes32 public constant ATTESTOR_ROLE = keccak256("ATTESTOR_ROLE");
    mapping(bytes32 => bytes32) public roots;
    mapping(bytes32 => uint256) public totalReserves;
    event RootUpdated(bytes32 indexed assetId, bytes32 root, uint256 total);
    constructor(address admin) { _grantRole(DEFAULT_ADMIN_ROLE, admin); }
    function updateRoot(bytes32 assetId, bytes32 root, uint256 total) external onlyRole(ATTESTOR_ROLE) {
        require(root != bytes32(0), "zero root forbidden");
        roots[assetId] = root; totalReserves[assetId] = total;
        emit RootUpdated(assetId, root, total);
    }
}

```

```

/// @title AssetVault
contract AssetVault is ReentrancyGuard, AccessControl {
    using SafeERC20 for IERC20;
    bytes32 public constant FACTORY_ROLE = keccak256("FACTORY_ROLE");
    IERC20 public immutable underlying;
    constructor(address u, address admin) {
        underlying = IERC20(u);
        _grantRole(DEFAULT_ADMIN_ROLE, admin);
        _grantRole(FACTORY_ROLE, admin);
    }
    function lock(address from, uint256 amt) external onlyRole(FACTORY_ROLE) nonReentrant {
        underlying.safeTransferFrom(from, address(this), amt);
    }
    function release(address to, uint256 amt) external onlyRole(FACTORY_ROLE) nonReentrant {
        underlying.safeTransfer(to, amt);
    }
}

```

```

/// @title FeeDistributor
contract FeeDistributor is AccessControl {
    address public aerarium;
    constructor(address admin, address a) { _grantRole(DEFAULT_ADMIN_ROLE, admin); aerarium = a; }
    receive() external payable { (bool ok,) = aerarium.call{value: msg.value}(""); }
}

```

```

/// @title HeirRegistry - iDEBT inheritance pattern integration
contract HeirRegistry {
    mapping(uint256 => bytes32) public heirRoot;
    event HeirRootSet(uint256 indexed tokenId, bytes32 root);
}

```

```

function setHeirRoot(uint256 tokenId, bytes32 root) external { heirRoot[tokenId] = root; }
function verifyHeir(uint256 tokenId, address heir, bytes32[] calldata p) external {
    return MerkleProof.verifyCalldata(p, heirRoot[tokenId], keccak256(abi.encode(tokenId, heir)));
}
}

```

```

/// @title WrappedAssetFactory

```

```

contract WrappedAssetFactory is AccessControl, ReentrancyGuard {
    bytes32 public constant DEPLOYER_ROLE = keccak256("DEPLOYER_ROLE");
    AttestationVerifier public immutable attest;
    RateLimiter public immutable rate;
    CircuitBreaker public immutable breaker;
    SenatusGate public immutable senatus;
    ProofOfReserves public immutable por;
    mapping(bytes32 => address) public wrappedOf;
    mapping(bytes32 => address) public vaultOf;
    event Deployed(bytes32 indexed assetId, address wrapped, address vault);
    event Wrapped(bytes32 indexed assetId, address indexed to, uint256 amount);
    event Unwrapped(bytes32 indexed assetId, address indexed from, uint256 amount);

    constructor(AttestationVerifier a, RateLimiter r, CircuitBreaker c, SenatusGate s, ProofOfReserves p) {
        attest = a; rate = r; breaker = c; senatus = s; por = p;
        _grantRole(DEFAULT_ADMIN_ROLE, admin); _grantRole(DEPLOYER_ROLE, admin);
    }

    function deployWrappedAsset(bytes32 id, string calldata n, string calldata s, uint256 d) public {
        require(wrappedOf[id] == address(0), "exists");
        v = address(new AssetVault(underlying, address(this)));
        w = address(new WrappedAsset(n, s, d, id, address(this), address(this), address(this)));
        wrappedOf[id] = w; vaultOf[id] = v;
        emit Deployed(id, w, v);
    }

    function wrap(bytes32 id, uint256 amt, bytes calldata vaa, bytes32 priceId, uint256 amt) public {
        require(!breaker.paused(), "paused");
        senatus.require_approval(keccak256(abi.encode("WRAP", id)));
        attest.verifyNttVaa(vaa);
        attest.verifyPythBound{value: msg.value}(priceId, refX18, upd);
        rate.consume(keccak256(abi.encode(id, "IN")), uint128(amt));
        WrappedAsset(wrappedOf[id]).mint(msg.sender, amt);
        emit Wrapped(id, msg.sender, amt);
    }

    function unwrap(bytes32 id, uint256 amt, bytes32 leaf, bytes32[] calldata proof) public {

```

```
require(!breaker.paused(), "paused");
senatus.require_approval(keccak256(abi.encode("UNWRAP", id)));
attest.verifyReserve(id, leaf, proof);
rate.consume(keccak256(abi.encode(id, "OUT")), uint128(amt));
WrappedAsset(wrappedOf[id]).burn(msg.sender, amt);
AssetVault(vaultOf[id]).release(msg.sender, amt);
emit Unwrapped(id, msg.sender, amt);
}
}
```

5.4 Algorand port (Algopy, Python ≥ 3.12)

python


```
# (c) 2026 BANKON – all rights reserved. Apache 2.0.
```

```
from algopy import ARC4Contract, BoxMap, Bytes, Global, Txn, UInt64, arc4, op, gtxn,
```

```
# ----- wrapped_asset.py -----
```

```
class WrappedAsset(ARC4Contract):
```

```
    asset_id: UInt64
```

```
    factory_app: UInt64
```

```
    total_supply: UInt64
```

```
    balances: BoxMap[arc4.Address, UInt64]
```

```
@arc4.abimethod(create="require")
```

```
def create(self, asset_id: UInt64, factory_app: UInt64) -> None:
```

```
    self.asset_id = asset_id
```

```
    self.factory_app = factory_app
```

```
    self.total_supply = UInt64(0)
```

```
@arc4.abimethod
```

```
def mint(self, to: arc4.Address, amount: UInt64) -> None:
```

```
    # Authorization: caller must be the factory app
```

```
    assert Txn.sender == Global.creator_address
```

```
    prev = self.balances.get(to, default=UInt64(0))
```

```
    self.balances[to] = prev + amount
```

```
    self.total_supply += amount
```

```
@arc4.abimethod
```

```
def burn(self, frm: arc4.Address, amount: UInt64) -> None:
```

```
    assert Txn.sender == Global.creator_address
```

```
    prev = self.balances[frm]
```

```
    assert prev >= amount
```

```
    self.balances[frm] = prev - amount
```

```
    self.total_supply -= amount
```

```
# ----- attestation_verifier.py -----
```

```
class AttestationVerifier(ARC4Contract):
```

```
    """Pyth-payload verifier built atop the existing Wormhole AVM core
```

```
    (wormhole_core / vaa_verify / TmplSig from wormhole-foundation/wormhole/algorand
```

```
    wormhole_core_app: UInt64
```

```
    pyth_emitter: Bytes # 32 bytes
```

```
    consumed: BoxMap[Bytes, UInt64]
```

```
@arc4.abimethod
```

```
def verify_vaa(self, vaa: Bytes) -> arc4.Bool:
```

```
h = op.sha512_256(vaa)
assert h not in self.consumed, "replay"
# delegate signature check to wormhole_core via inner app call group
itxn.ApplicationCall(app_id=self.wormhole_core_app, app_args=[b"verifyVAA",
self.consumed[h] = UInt64(1)
return arc4.Bool(True)
```

```
# ----- rate_limiter.py -----
```

```
class RateLimiter(ARC4Contract):
    capacity: BoxMap[Bytes, UInt64]
    rate: BoxMap[Bytes, UInt64]
    tokens: BoxMap[Bytes, UInt64]
    last: BoxMap[Bytes, UInt64]

    @arc4.abimethod
    def consume(self, key: Bytes, amount: UInt64) -> None:
        now = Global.latest_timestamp
        dt = now - self.last[key]
        refill = dt * self.rate[key]
        new_tok = self.tokens[key] + refill
        cap = self.capacity[key]
        if new_tok > cap:
            new_tok = cap
        assert new_tok >= amount, "RateLimited"
        self.tokens[key] = new_tok - amount
        self.last[key] = now
```

```
# ----- circuit_breaker.py -----
```

```
class CircuitBreaker(ARC4Contract):
    paused: UInt64
    @arc4.abimethod
    def trip(self) -> None:
        assert Txn.sender == Global.creator_address
        self.paused = UInt64(1)
    @arc4.abimethod
    def reset(self) -> None:
        assert Txn.sender == Global.creator_address
        self.paused = UInt64(0)
```

```
# ----- senatus_gate.py -----
```

```
class SenatusGate(ARC4Contract):
```

```

senatus_app: UInt64
@arc4.abimethod
def require_approval(self, op_id: Bytes) -> None:
    itxn.ApplicationCall(app_id=self.senatus_app, app_args=[b"check", op_id]).su

```

----- proof_of_reserves.py -----

```

class ProofOfReserves(ARC4Contract):
    roots: BoxMap[Bytes, Bytes]
    totals: BoxMap[Bytes, UInt64]
    @arc4.abimethod
    def update_root(self, asset_id: Bytes, root: Bytes, total: UInt64) -> None:
        assert Txn.sender == Global.creator_address
        assert root != Bytes(b"\x00" * 32), "zero root forbidden"
        self.roots[asset_id] = root
        self.totals[asset_id] = total

```

----- asset_vault.py -----

```

class AssetVault(ARC4Contract):
    asset_id: UInt64
    @arc4.abimethod
    def lock(self, axfer: gtxn.AssetTransferTransaction) -> None:
        assert axfer.xfer_asset == self.asset_id
        assert axfer.asset_receiver == Global.current_application_address
    @arc4.abimethod
    def release(self, to: arc4.Address, amount: UInt64) -> None:
        assert Txn.sender == Global.creator_address
        itxn.AssetTransfer(xfer_asset=self.asset_id, asset_receiver=to.native, asset

```

----- wrapped_asset_factory.py -----

```

class WrappedAssetFactory(ARC4Contract):
    attest_app: UInt64
    rate_app: UInt64
    breaker_app: UInt64
    senatus_app: UInt64
    por_app: UInt64
    wrapped_of: BoxMap[Bytes, UInt64]
    vault_of: BoxMap[Bytes, UInt64]

    @arc4.abimethod
    def wrap(self, asset_id: Bytes, amount: UInt64, vaa: Bytes) -> None:
        # 1. breaker

```

```

    paused = op.AppGlobal.get_ex_uint64(self.breaker_app, b"paused")[0]
    assert paused == 0, "paused"
    # 2. senatus
    itxn.ApplicationCall(app_id=self.senatus_app, app_args=[b"check", b"WRAP", a
    # 3. wormhole+pyth attestation
    itxn.ApplicationCall(app_id=self.attest_app, app_args=[b"verify_vaa", vaa]).
    # 4. rate limit
    itxn.ApplicationCall(app_id=self.rate_app, app_args=[b"consume", asset_id +
    # 5. mint
    itxn.ApplicationCall(app_id=self.wrapped_of[asset_id], app_args=[b"mint", Tx

```

5.5 x402 Algorand payment integration (Parsec wallet abstraction)

Every state-changing PWAFAPI call is metered through github.com/GoPlausible/x402-avm, abstracted behind the Parsec wallet layer. The middleware on mindx.pythai.net:

```

ts

// mindx.pythai.net/server.ts
import { Hono } from 'hono';
import { paymentMiddleware } from '@x402-avm/hono';
import { registerExactAvmScheme } from '@x402-avm/avm';

const app = new Hono();
app.use('*', paymentMiddleware({
  scheme: 'exact',
  network: 'algorand:wGHE2Pwvdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=', // CAIP-2 mainnet
  payTo: process.env.AERARIUM_ALGO_ADDR!,
  asset: { type: 'asa', id: 31566704 }, // USDC mainnet
  price: { wrap_asset: '0.10', unwrap_asset: '0.10', check_reserves: '0.01' },
  facilitatorUrl: 'https://facilitator.parsec.bankon.pythai.net',
}));
registerExactAvmScheme(app);

```

The on-chain settlement is a 2-txn atomic group: the user signs an [AssetTransferTxn](#) of USDC (ASA 31566704, mainnet [USDC_MAINNET_ASA_ID](#) constant from [x402-avm](#)) to [AERARIUM_ALGO_ADDR](#); the facilitator signs a fee-payer transaction (amount=0, fee=2000) to cover the user's pooled fee. There is no custom ARC4 ABI contract on chain — the protocol uses bare AVM atomic groups consistent with the [@x402-avm/avm](#) [ExactAvmScheme](#) and [FacilitatorAvmSigner](#) interfaces. The Parsec wallet layer handles signer routing (Pera, Defly, MyAlgo, WalletConnect) so the user experience is single-click. [GitHub](#) [GitHub](#)

5.6 Chain mapping registry & CAIP-2 routing

PWAF consults <https://agenticplace.pythai.net/allchain.html> as the CAIP-2 indexed registry of supported chains (genesis hashes, RPCs, x402 facilitator addresses, openBDK relay/validator topology). Algorand mainnet is [algorand:wGHE2Pwddvd7S12BL5Fa0P20EGYesN73ktiC1qzkkkit8=](#). Arc Testnet (Chain ID 5042002, USDC-native gas) is [eip155:5042002](#). Ethereum mainnet is [eip155:1](#). The factory routes by CAIP-2 selector: [eip155:*](#) → CCIP/NTT path; [algorand:*](#) → AVM inner-app-call path. [GitHub](#)

5.7 AgenticPlace + mindX (AI SDK v6)

```
ts

import { tool, generateText, ToolLoopAgent } from 'ai';
import { z } from 'zod';

export const wrap_asset = tool({
  description: 'Wrap an underlying asset into a PWAF wrapped ERC-20 / ASA.',
  parameters: z.object({ asset_id: z.string(), amount: z.string(), dest_chain_caip2: z.string() }),
  execute: async (args) => fetch('https://mindx.pythai.net/v1/wrap', { method: 'POST', body: JSON.stringify(args) });
});

export const unwrap_asset = tool({ /* ... */ });
export const check_reserves = tool({ /* ... */ });
export const query_rate_limit = tool({ /* ... */ });
export const propose_senatus_veto = tool({ /* ... */ });

export const pwafAgent = new ToolLoopAgent({
  tools: { wrap_asset, unwrap_asset, check_reserves, query_rate_limit, propose_senatus_veto },
  model: 'pythai/mindx-1',
});
```

Tools are also exposed over MCP for non-AI-SDK clients.

5.8 BANKON identity layer

Every wrap/unwrap requires a BONAFIDE attestation from <https://bankon.pythai.net>. Flow: user authenticates via Pera/Defly; bankon mints a short-lived attestation JWT signed by the user's Tessera identity NFT key (ARC-52 DID document on Algorand, EIP-712 typed signature when used on EVM); the JWT is attached to the wrap request; [SenatusGate.require_approval](#) rejects if the attestation is missing or revoked (revocations recorded in [Censura](#)).

5.9 DAIO deployment plan

Phase 1 (constitutional, Algorand mainnet): deploy the BONAFAIDE 9-contract suite — Genius, BonaToken, Tabularium, Fides, SponsioPactum, Censura, Senatus, Tessera, Aerarium/Curia — plus PWAF AVM apps (`wrapped_asset_factory`, `attestation_verifier`, `rate_limiter`, `circuit_breaker`, `senatus_gate`, `proof_of_reserves`, `asset_vault`, `wrapped_asset`). Phase 2 (economic, Arc Testnet 5042002 staging then Ethereum mainnet): deploy PWAF Solidity contracts via Foundry script using deterministic CREATE2. Phase 3 (openBDK bridge): instantiate the 1 Relayer + 3 Validator BFT topology bridging Algorand ↔ Arc Testnet ↔ Ethereum, preserving the canonical Polygon L1Escrow + L2MinterBurner + NativeConverter pattern. Phase 4 (multi-chain mainnet): roll out per-chain PWAF deployments registered under CCIP (`TokenAdminRegistry`) and Wormhole (`NttManager`), with rate limits scaled to expected daily flow + 2σ .

5.10 Foundry test suite (regression coverage of Part 2 classes)

solidity

```

// test/Pwaf.t.sol
// SPDX-License-Identifier: Apache-2.0
pragma solidity 0.8.26;

import "forge-std/Test.sol";
import "../src/WrappedAssetFactory.sol";

contract PwafTest is Test {
    WrappedAssetFactory factory;
    AttestationVerifier attest;
    RateLimiter rate;
    CircuitBreaker breaker;
    SenatusGate senatus;
    ProofOfReserves por;
    address constant ADMIN = address(0xA11CE);
    bytes32 constant ASSET_BTC = keccak256("BTC");

    function setUp() public {
        attest = new AttestationVerifier(address(0xWH), address(0xPY), bytes32(uint256(1)));
        rate = new RateLimiter();
        breaker = new CircuitBreaker(ADMIN);
        senatus = new SenatusGate(address(0xSEN));
        por = new ProofOfReserves(ADMIN);
        factory = new WrappedAssetFactory(attest, rate, breaker, senatus, por, ADMIN);
    }

    /// @notice Invariant: total wrapped supply across chains == total locked reserves
    function invariant_supplyEqualsReserves() public view {
        // pull cross-chain supply from openBDK reserve manager; abbreviated
    }

    /// @notice Regression: Nomad zero-root must be impossible.
    function test_Revert_ZeroReserveRoot() public {
        vm.prank(ADMIN);
        vm.expectRevert(bytes("zero root forbidden"));
        por.updateRoot(ASSET_BTC, bytes32(0), 0);
    }

    /// @notice Regression: Wormhole sysvar bypass – strictly require Pyth emitter.
    function test_RejectWrongEmitter() public {
        // craft a VAA with wrong emitterChainId; expect revert via InvalidEmitter
    }
}

```

```

/// @notice Regression: Poly Network – no privileged target reachable from cross
function test_NoPrivilegedTargetFromCrossChain() public {
    // verify AttestationVerifier has no externally callable admin writer
}

/// @notice Regression: KelpDAO single DVN – Wormhole 13-of-19 quorum hard-coded
function test_HardcodedQuorum() public {
    // attempt to set fewer than 13 guardians; expect revert (Wormhole governance)
}

/// @notice Fuzz: rate limiter never lets through more than capacity + drift per
function testFuzz_RateLimiterBounds(uint128 cap, uint128 ratePerSec, uint16 calls) public {
    vm.assume(cap > 0 && cap < type(uint64).max);
    vm.assume(ratePerSec < cap);
    bytes32 key = keccak256("BTC_IN");
    rate.configure(key, cap, ratePerSec);
    uint256 total;
    for (uint256 i = 0; i < calls; i++) {
        try rate.consume(key, 1) { total++; } catch { break; }
    }
    assertLe(total, uint256(cap) + uint256(ratePerSec) * block.timestamp);
}

/// @notice Fork test against Ethereum mainnet Pyth contract.
function test_ForkPythPriceFreshness() public {
    vm.createSelectFork("mainnet");
    // call IPyth(0x4305FB66699C3B2702D4d05CF36551390A4c69C6) and assert getValid
}

/// @notice Regression: BNB IAVL – use only OZ MerkleProof, depth-bounded.
function test_OzMerkleOnly() public {
    bytes32[] memory proof = new bytes32[](0);
    bytes32 leaf = keccak256("x");
    assertFalse(MerkleProof.verifyCalldata(proof, bytes32(uint256(1)), leaf));
}
}

```

Run with `forge test -vvv` and `forge coverage --report summary`. Solidity 0.8.26, OZ v5, `forge-std`.

5.11 Cryptoeconomic security analysis (per-class)

Mapping each Part 2 exploit to the PWAF defense layer that catches it:

Ronin/Harmony/Multichain/Orbit/HTX/Mixin (validator/MPC compromise) → defeated by 13-of-19 Wormhole + 5-of-7 Senatus + Pyth deviation + rate limit; even total compromise of Wormhole still trips Pyth (price out of envelope) and Senatus veto. Wormhole sysvar bypass → mitigated by Pyth emitter strict check + Senatus + Merkle reserve check, plus the post-audit Wormhole code now validates sysvar accounts on Solana. Nomad zero-root → explicit `require(root != bytes32(0))` in both `AttestationVerifier` and `ProofOfReserves`; per-VAA hash replay tracking. BNB IAVL → PWAF does not use IAVL; only depth-bounded OZ `MerkleProof.verifyCalldata`. Poly Network privileged target → factory exposes no admin writer reachable from cross-chain; mint role gated to the factory, factory callable only via wrap/unwrap which require Pyth + Senatus. KelpDAO single-DVN → hard-coded 13-of-19 not configurable.

5.12 Comparison table

Property	kBTC on LayerZero (pre-May 2026)	kBTC on CCIP (May 2026+)	PWAF (Pyth+Wormhole+BONAFIDE)
Verifier model	App-configurable DVN set (1-N)	DON ≥ 16 + independent RMN	Wormhole 13/19 + Pyth deviation + Senatus 5/7
Min attacker quorum	1 (Kelp config)	$\geq t$ DON + defeat RMN	≥ 13 Guardians AND Pyth AND Senatus AND RL
Sovereignty	LayerZero Labs governance	Chainlink Labs governance	DAIO + Senatus DAO
Open / closed	Permissionless DVN registry	Permissioned operator set	Open Wormhole + open Pyth + sovereign BONAFIDE
Rate limits	Optional per OApp	Native per token/lane	Native per asset/direction/epoch + EMA refill
Proof-of- reserves	Off-chain at Kraken	Off-chain at Kraken + CCIP PoR feed	On-chain Merkle root, Pyth-attested
Price sanity check	None	None	Pyth deviation envelope
Identity layer	None	None	BONAFIDE Tessera/Fides attestation
Fee metering	None	LINK on send	x402 per call (USDC ASA 31566704)
Algorand support	No	No (EVM only today)	Native (Algopy + Wormhole AVM core)

5.13 Migration path for existing wrapped assets

For WBTC, kBTC, or any existing wrapper: (1) deploy a PWAF `WrappedAsset` and `AssetVault` on each destination; (2) deploy a one-way upgrade contract accepting legacy tokens 1:1 for PWAF tokens; for kBTC, Kraken signs a migration NTT VAA in "locking" mode against the legacy reserve; (3) repoint CCIP `TokenAdminRegistry` from the legacy pool's token to the new PWAF token; (4) deprecate the legacy token after a 6-month grace.

Part 6 — Code requirements (compliance summary)

All Solidity is `pragma solidity 0.8.26`, OpenZeppelin v5 imports, NatSpec on external functions, Apache 2.0 header `(c) 2026 BANKON – all rights reserved`. Foundry: `forge-std/Test.sol`, `forge test`, `forge coverage`. Snake_case Algopy filenames; PascalCase

Solidity contracts. Algopy uses `ARC4Contract`, `@arc4.abimethod`, `BoxMap`. Python ≥ 3.12 typing throughout. Cypherpunk2048 flat snake_case repo layout.

Part 7 — References (inline hyperlinks)

Kraken-CCIP migration: Chainlink Today (<https://chainlinktoday.com/kraken-upgrades-to-chainlink-ccip-for-kbtc-and-all-future-wrapped-assets/>), CoinDesk (<https://www.coindesk.com/business/2026/05/14/kraken-to-replace-layerzero-with-chainlink-to-bridge-assets-across-blockchains>), The Block (<https://www.theblock.co/post/401368/chainlink-ccip-gains-over-2-5-billion-tvl-from-protocols-migrating-layerzero-kraken-bitcoin-latest>), Crypto Briefing (<https://cryptobriefing.com/kraken-chainlink-ccip-integration/>), Blockonomi (<https://blockonomi.com/kraken-migrates-kbtc-to-chainlink-ccip-as-layerzero-exodus-grows>). LayerZero kBTC OFT integration (<https://layerzero.network/blog/kraken-integrates-layerzero-technology-for-kbtc>). Chainlink CCT docs: <https://docs.chain.link/ccip/concepts/cross-chain-tokens>, <https://docs.chain.link/ccip/concepts/cross-chain-token/evm/architecture>, <https://docs.chain.link/ccip/concepts/cross-chain-token/evm/token-pools>, <https://docs.chain.link/ccip/api-reference/evm/v1.6.0/burn-mint-token-pool>, <https://docs.chain.link/ccip/tutorials/cross-chain-tokens>, <https://blog.chain.link/empowering-developers-with-the-cct-standard/>, <https://chain.link/cross-chain>. Chainlink Foundry local simulator example: <https://docs.chain.link/chainlink-local/build/ccip/foundry/cct-burn-and-mint-fork>. LayerZero DVN docs: <https://docs.layerzero.network/v2/workers/off-chain/build-dvns>, <https://layerzero.network/blog/layerzero-v2-explaining-dvns>, GitHub <https://github.com/LayerZero-Labs/LayerZero-v2>. KelpDAO postmortem: <https://www.blockaid.io/blog/how-a-single-layerzero-dvn-compromise-drained-292m-from-kelpdao>, <https://blog.reactive.network/defi-hack-postmortem-why-reactive-contracts-are-the-bridge-security-model-defi-needs/>. Wormhole docs: <https://wormhole.com/docs/protocol/infrastructure/guardians/>, <https://wormhole.com/docs/protocol/infrastructure/vaas/>, <https://wormhole.com/docs/products/token-transfers/native-token-transfers/concepts/architecture/>, <https://wormhole.com/docs/protocol/security/>, <https://wormhole.com/docs/products/reference/glossary/>, <https://wormhole.com/blog/deep-dive-wormhole-native-token-transfers-ntt>, <https://wormhole.com/blog/wormhole-101-guardians>. Wormhole NTT GitHub <https://github.com/wormhole-foundation/native-token-transfers>. Wormhole Algorand core <https://github.com/wormhole-foundation/wormhole/tree/main/algorand>. Wormhole vulnerability disclosure (Guardian set expiration): <https://marcohexor.com/wormhole-one-key-vulnerability/>. Pyth docs: <https://docs.pyth.network/price-feeds/core/api-instances-and-providers/hermes>,

[feeds/core/contract-addresses](#)), (<https://docs.pyth.network/entropy>), (<https://docs.pyth.network/entropy/protocol-design>), (<https://docs.pyth.network/entropy/contract-addresses>), (<https://www.pyth.network/blog/pyth-entropy-random-number-generation-for-blockchain-apps>), (<https://www.pyth.network/entropy>). Pyth Lazer: (<https://www.theblock.co/post/334927/pyth-launches-new-oracle-lazer-to-offer-price-feeds-to-latency-sensitive-apps>), (<https://blockchain.news/flashnews/pyth-network-lazer-oracle-upgrade-slashes-amm-latency-for-lightning-fast-crypto-trading>), (<https://www.rootdata.com/news/264042>). Pyth SDK Solidity: (<https://github.com/pyth-network/pyth-sdk-solidity/blob/main/IPyth.sol>), (<https://github.com/pyth-network/pyth-sdk-solidity/blob/main/PythStructs.sol>). Pyth crosschain monorepo: (<https://github.com/pyth-network/pyth-crosschain>). Pyth overview: (<https://messari.io/report/understanding-pyth-network-a-comprehensive-overview>). Wormhole overview: (<https://messari.io/report/understanding-wormhole-a-comprehensive-overview>). Bridge exploit postmortems: Ronin (<https://roninchain.com/blog/posts/back-to-building-ronin-security-breach-6513cc78a5edc1001b03c364>), (https://github.com/tayvano/lazarus-bluenoroff-research/blob/main/hacks-and-thefts/ronin_bridge.md), (<https://grvt.io/blog/crypto-history-ronin-bridge-hack/>); Wormhole (<https://www.halborn.com/blog/post/explained-the-wormhole-hack-february-2022>), (<https://www.certik.com/resources/blog/wormhole-bridge-exploit-incident-analysis>), (<https://blocksec.com/blog/revisiting-the-wormhole-attacks>); Nomad (<https://www.certik.com/blog/nomad-bridge-exploit-incident-analysis>), (<https://medium.com/immunefi/hack-analysis-nomad-bridge-august-2022-5aa63d53814a>), (<https://cloud.google.com/blog/topics/threat-intelligence/dissecting-nomad-bridge-hack>), (<https://nomoslabs.io/blog/nomad-bridge-exploit-technical-breakdown-190m-hack>); Poly Network (<https://blog.kraken.com/product/security/abusing-smart-contracts-to-steal-600-million-how-the-poly-network-hack-actually-happened>), (<https://research.kudelskisecurity.com/2021/08/12/the-poly-network-hack-explained/>), (<https://rekt.news/polynetwork-rekt>); Harmony (<https://www.elliptic.co/blog/analysis/over-1-billion-stolen-from-bridges-so-far-in-2022-as-harmony-s-horizon-bridge-becomes-latest-victim-in-100-million-hack/>), (<https://www.merklescience.com/blog/hack-track-analysis-of-harmonys-horizon-bridge-exploit>); BNB Token Hub (<https://www.halborn.com/blog/post/explained-the-bnb-chain-hack-october-2022>), (<https://medium.com/immunefi/hack-analysis-binance-bridge-october-2022-2876d39247c1>), (<https://www.coinbase.com/blog/bsc-token-hub-compromise-investigation-and-analysis-8>), (<https://www.nansen.ai/research/bnb-chains-cross-chain-bridge-exploit-explained>), (<https://swarm.ptsecurity.com/binance-smart-chain-token-bridge-hack/>), (<https://www.quillaudits.com/blog/hack-analysis/bsc-token-hub-bridge-hack>); Multichain (<https://www.chainalysis.com/blog/multichain-exploit-july-2023/>); Heco (<https://hacken.io/insights/heco-bridge-hack-explained/>); Orbit

<https://www.coindesk.com/business/2024/01/02/orbit-chain-loses-81m-in-cross-chain-bridge-exploit>, <https://cointelegraph.com/news/orbit-bridge-hack-pushes-december-crypto-losses-100m>). Etherscan kBTC token <https://etherscan.io/token/0x73E0C0d45E048D25Fc26Fa3159b0aA04BfA4Db98>). GoPlausible x402-avm <https://github.com/GoPlausible/x402-avm> plus the documentation set under <https://github.com/GoPlausible/.github/tree/main/profile/algoland-x402-documentation> and npm <https://www.npmjs.com/package/@x402-avm/core>. Foundry book <https://book.getfoundry.sh/>. OpenZeppelin v5 <https://github.com/OpenZeppelin/openzeppelin-contracts>. Axelar overview <https://messari.io/report/understanding-axelar-a-comprehensive-overview>. EIPs/ARCs are linked via standard registries: EIP-7281 (xERC20), EIP-7857 (INFT), ARC-19, ARC-69, ARC-52, ARC-200.

Recommendations

Stage 1 — Build the Algorand sovereign layer first. Deploy the BONAFIDE 9-contract suite on Algorand mainnet, plus the PWAFAVM apps (`wrapped_asset_factory`, `attestation_verifier`, `rate_limiter`, `circuit_breaker`, `senatus_gate`, `proof_of_reserves`, `asset_vault`, `wrapped_asset`). Wire `attestation_verifier` to the existing Wormhole Algorand core. Establish the Senatus 5-of-7 sovereign quorum with Tessera-bearing members. Threshold to advance: a full Foundry-equivalent Algopy `algokit test` suite passes invariant tests `total_supply == total_reserves` over 10,000 fuzz iterations, and a Pera-Wallet signed BONAFIDE attestation can drive a wrap and an unwrap end-to-end on Algorand testnet.

Stage 2 — Build the EVM economic layer on Arc Testnet (Chain ID 5042002) using USDC-native gas, deploy the Solidity factory and per-asset wrappers via Foundry CREATE2, and stand up x402-avm metering through `mindx.pythai.net` so every API call is gated by USDC ASA 31566704 settlement on Algorand. Threshold to advance: `forge coverage` $\geq 95\%$ on factory + verifier + rate limiter + circuit breaker + reserves; all Part 2 regression tests pass; one stETH wrap end-to-end on Arc Testnet with Pyth deviation envelope active.

Stage 3 — Wire openBDK as the BFT bridge (1 Relay + 3 Validators, canonical Polygon L1Escrow + L2MinterBurner + NativeConverter pattern preserved) between Algorand and Arc Testnet, then between Arc Testnet and Ethereum. Threshold to advance: a \$10M-equivalent stress test under simulated DVN-style attack fails to mint counterfeit tokens because Pyth deviation + Senatus + rate limiter trip before settlement.

Stage 4 — Mainnet rollout: register PWAFAVM wrappers under CCIP `TokenAdminRegistry` (so kBTC and any other CCT can interoperate at the perimeter) and Wormhole `NttManager` (for the core defense-in-depth path). Migrate first asset (suggest a small-cap RWA, then a BTC wrapper). Threshold to scale further: 30 days without a single Senatus veto, Pyth deviation never exceeds 50 bps in normal flow, rate limiter capacity utilization $< 60\%$ of design.

Stage 5 — Open the factory to external issuers: publish the

`WrappedAssetFactory.deployWrappedAsset` interface, charge a one-time x402-metered deployment fee paid to AERARIUM, and require a BONAFIDE-Senatus quorum vote for each new asset listing. This is the "factory" generalization that turns PWAF from a single-asset platform into a sovereign Wrapped-Asset-as-a-Service.

Benchmarks that would change these recommendations: if Chainlink open-sources the RMN codebase and its DON membership becomes provably permissionless (which would close the sovereignty gap), CCIP-only operation should be reconsidered; if Wormhole publishes Guardian compromise insurance or moves to economic stake, the PWAF Senatus layer can be simplified; if Pyth ships a canonical Algorand mainnet deployment, the PWAF `attestation_verifier` can drop its custom Pyth-payload parser and use the official client.

Caveats

Several search results referenced future-tense or speculative reporting about Kraken's CCIP timeline and Q1 2026 CCIP transfer growth (cited as 78% QoQ, 319% YoY, 165% token count growth); these come from aggregator coverage and should be treated as marketing-inflected rather than primary. Pyth is not deployed on Algorand mainnet as of May 2026; PWAF's Algorand layer therefore implements a Pyth-payload verifier on top of the existing Wormhole AVM core rather than wrapping an official Pyth Algorand client. The KelpDAO attribution to Lazarus is per incident reports cited by Blockaid and The Block; the LayerZero "made a mistake" statement is paraphrased from coverage. Kraken's actual deployed CCIP `TokenAdminRegistry` addresses for kBTC were not yet indexed in the Chainlink CCIP Directory at the time of writing; engineers implementing the migration code should consult <https://docs.chain.link/ccip/directory> for live values. Loss figures vary across sources (Ronin \$625M vs \$551M; BNB \$570M vs \$586M) due to intra-day ETH price movements during events; this report uses the most widely-cited figure with the alternative noted. The Axelar active-validator-set claim of exactly 75 is per Messari's *Understanding Axelar* overview ("Only the top 75 validators are in the active set, a parameter that can be adjusted through onchain governance") and may change via governance. Pyth's publisher count is per Pyth's own Network KPIs page ("138 first-party publishers" / "Over 120 financial institutions") and the precise number is updated continuously. The wormhole expired-Guardian-set vulnerability disclosure cited (one-key Wormchain bug, Marco Hextor / ImmuneFi) was patched but illustrates the criticality of strict expiration checks in the Guardian set machinery — a property PWAF's `AttestationVerifier` inherits by delegation to upstream `parseAndVerifyVM`. Finally, GoPlausible's `x402-avm` does not expose an on-chain ARC4 ABI contract — settlement is a bare AVM 2-txn atomic group of `AssetTransferTxn` (USDC ASA 31566704) + facilitator fee-payer — so PWAF should not assume the existence of an on-chain x402 settlement contract; the "API side" of the pattern is the canonical integration surface.